

Problem Set 4: Querying Strings

For this assignment, you will use Python and Visual Studio Code (VSC) to create a computer program. Submit a Python program script file with the solutions (file extension is py) in Blackboard by Sunday at 11:59 PM. There are many facets to working with strings in data. This includes comparing strings or finding specific characters, words, or phrases. More advanced usage of textual data includes authorship analysis with the imposter algorithm or assessing sentiment in business reviews. Another example is the count of keywords in search engine page ranking. At the end of these instructions, there is some additional information regarding testing.

Script Requirements

When writing and saving your programming files, ensure that you heed the instructions. For all programming coursework, specific things must be placed in the programming code. The beginning of the script file must include the assignment name, the purpose of the programming code, the date it was written or revised, and who wrote the code. These comments can be written as line comments or a commenting block. Figure 1 depicts an example of what this might look like in your file. When saving the program for the first problem, name the file **assignment_4_1.py**. The second, third, and fourth problems shall be saved with the file names **assignment_4_2.py**, **assignment_4_3.py**, and **assignment4_4.py**, respectively. When you have completed and tested your script files, submit all of the script files for this assignment in Blackboard.

As it has already been established, all analysis begins with a problem to solve, a question to answer, or something to prove (you might call this a hypothesis). The following information establishes the requirements of the program and some hints for completing this work. There are four problems that you need to solve for this assignment.

Figure 1

Content Needed in Every Script File

```
"""
Assignment 4 Problem 1: find and count matching strings
September 22, 2022
John Smith
"""
```

Note. This contains the name of the assignment (Assignment 4 Problem 1), purpose of this assignment (find and count matching strings), date (September 22, 2022), and the name of the developer (John Smith).

Problem One

Currently, you have no way to count the number of times a substring matches within another string. To solve this problem, you will create two functions: **countSubstrMatches** and **countSubstrRecursive**. Both of these functions will take two arguments: a string to search and the substring you want to find.

```
def countSubstrMatches(srch_str, sub_str):
    """
    Describe what the function does, what type of information goes
    into the arguments, and what is returned.
    """

def countSubstrRecursive(srch_str, sub_str):
    """
    Describe what the function does, what type of information goes
    into the arguments, and what is returned.
    """
```

Your functions will both use the method **find** to identify the matches. The first function will search iteratively. The second function will search recursively. While you are working on creating these functions, you must import the string library, understand the use of **find**, and

Figure 2

*Examples Using the **find** Method in Python*

```
>>>"atatattta".find("ata")      # search for substring ata
0
>>>"atatattta".find("ttt")
5
>>>"atatattta".find("ata", 1)  # search for ata after index 0
2
>>>"python".find("Python")     # don't forget! case matters
-1
```

Note. This is an example of what you may see when using the find method in the terminal after importing the strings library. The text in yellow reflects the output from the program.

The text in white reflects the user's input.

understand the difference between iteration and recursion. After the comments needed in every script file, add **from string import *** to the script file so that you have access to the built-in string functions in base Python. When using the method **find**, you must have a string to search and a substring to find. There are optional arguments, as well. Optionally, you can also provide the start and stop index values. When you use this method, it returns a number. If the value return is a negative one, no matches were found. Alternatively, the returned value is the index position of the first match. There are examples of the usage, input, and output of this method shown in Figure 2. Ensure assumptions are not made regarding the string that is going to be searched. This string could be a word, phrase, novel, or nonsense. When you work on the function that will search recursively, think about how you can break this problem down into smaller pieces. Adding an optional argument to the function might be helpful. If you're wondering what optional argument would be useful here, consider how you processed the string in the iterative function.

Figure 3

Examples of Expected Output for the Solution to Problem 2

```
allMatchesIndices("atatattta", "ata")
(1,)
allMatchesIndices("atatatatta", "ata")
(0, 2, 4)
allMatchesIndices("atattatta", "Python")
()
```

Note. This is an example of what you may see when using the find method in the terminal after importing the strings library. The text in yellow reflects the output from the program. The text in white reflects the function call.

Problem Two

The second problem is that you do not have a function that will collect the start positions of every match between a substring and a string. For this solution, you will create one function named **allMatchesIndices**. This function will require two arguments, the string you need to search and the substring you want to find. The function shall return a tuple of the start indices of all matches.

```
def allMatchesIndices(srch_str, sub_str):
    """
    Describe what the function does, what type of information goes
    into the arguments, and what is returned.
    """
```

Keep in mind indices start at zero. An example of the expected output is shown in Figure 3.

Since the required output is a tuple, using an iterative method will probably be a far less complicated method for finding a solution.

Problem Three

The third problem is that you need a way to search for fuzzy matches. For this solution, your goal is to find where all but one character in a substring match by creating a function named **fuzzyMatching**. For example, suppose the substring is **abcd**. In that case, positive matches include **abc***, **ab*d**, **a*cd**, and ***bcd** (where ***** represents a wild card that any character can substitute). The function **fuzzyMatching** has three required arguments. The first argument is a tuple of matching start positions for the first substring (the tuple output of the function you created to solve the second problem). The second required argument is a tuple of the second substring. The length of the first substring is the third argument. The function returns a tuple of start positions of all fuzzy matches.

```
def fuzzyMatching(subOne, subTwo):  
    """  
    Describe what the function does, what type of information goes  
    into the arguments, and what is returned.  
    """
```

This is an example of what information is sent to this function and why it's necessary. If you wanted to find the fuzzy matches with **"a*cd"** as the substring, then **"a"** and **"cd"** are the substrings. The tuple of the first substring can be collected by using the solution to problem two, **allMatchesIndices("abcdefgabad", "a")** which returns the tuple **(0, 7, 9)**. The second substring is **allMatchesIndices("abcdefgabad", "cd")** which returns the tuple **(2,)**. The first substring's length can be collected using **len("a")**, which returns **1**. These are the arguments for the function that solve this problem.

```
fuzzyMatching((0, 7, 9), (2, ), len("a"))
```

Add the length of the first substring **"a"** and the number of wild cards to each of the start positions in the first tuple (this would equate to $0 + 1 + 1 = 2$, $7 + 1 + 1 = 9$, and $9 + 1$

+ 1 = 11), if any of the totals are equivalent to a value found in the second tuple, then keep the corresponding value in the first tuple (in this example, since $0 + 1 + 1 = 2$ and there is a 2 in the second tuple, retain the 0 from the first tuple). There is only one match (fuzzy or otherwise) in the string. The returned tuple for the example is (0,). Position zero in the string is the starting point of the only match (abcd is fuzzy matched to a*cd).

Problem Four

The fourth and final problem is that the solution to problem three returns exact and fuzzy matches. The solution to this program is to create a function named **fuzzyMatchesOnly** that only returns string matches that are not an exact match. This function has two required arguments, the string that will be searched and a substring that you want to find. The output of this function is a tuple that provides the start position of fuzzy matches where one character is incorrect. (The returned tuple will not contain the start positions of exact matches.) Considering the solutions to problems 2 and 3, you may find that you have already done most of the work to solve this problem.

```
def fuzzyMatchesOnly(srch_str, sub_str):  
    """  
        Describe what the function does, what type of information goes  
        into the arguments, and what is returned.  
    """
```

Testing Code

Before submitting your tested solutions, ensure that you comment out all of the code you used to test your files. However, do not delete the testing code. Part of your grade is based on the testing you conduct.